# سكريبت العرض

سلايد 1 إلى 4 (الـ hook، المشكلة، الحل، الـ Architecture) | المقدّم: عمر محمد حسني | المدة المتوقعة: 4 إلى 5 دقايق

> الجمل اللي على الشمال بخط أخضر هي جمل الربط بين الأفكار. الـ hook في أول سلايد لازم يتقال بهدوء وبعد وقفة قصيرة، من غير ما تبص في الورقة.

## سلايد 1: العنوان (الـ hook)

صباح الخير يا دكتور، وصباح الخير يا جماعة.

قبل ما نبدأ، تعالوا نلعب لعبة تخمين صغيرة. **(وقفة قصيرة)**

عندنا حملة إعلانية اسمها **eid_sale_1**. تقرير الإعلان بيقول: كل **1 جنيه** صرفناه رجّع لنا **21 جنيه ونص**. حملة ممتازة، صح؟ خمّنوا الرقم الحقيقي بعد ما نشيل الأوردرات اللي اتلغت ورجعت.

الرقم الحقيقي **11.67**. يعني التقرير كان شايف الحملة أحسن منها بالضعف تقريبًا. والسبب إن في مصر كتير من الأوردرات كاش عند الاستلام، فيتسجل أوردر، وبعدين العميل بيلغيه أو مبيستلمهوش. وده على مستوى المتجر كله: اتسجّل **2,410,979 جنيه** مبيعات، والمتسلّم فعلًا **1,914,103**. يعني حوالي **نص مليون جنيه** كانوا على الورق بس.

> ولسه في حاجة أسوأ. حوالي **واحد من كل عشرة clicks** على الإعلانات وصل لمنتج مكانه في المخزن فاضي. العميل اتصرف عليه فلوس عشان يوصل، وبعدين لقى "غير متاح". والنظام التقليدي ما بيقولش لحد تاني يوم.

(ملاحظة بصدق: الأرقام دي من بيانات المحاكاة بتاعتنا، مش من متجر حقيقي. بس المحاكاة مبنية على نفس سلوك السوق المصري، وده بالظبط اللي بيحصل عند أي متجر.)

أنا عمر، ومعايا رفيدة رمضان، وفاطمة معتز، وأسماء صالح، وسمية محمود. واحنا بنيّنا **FunnelStream**: نظام بيشوف الحاجات دي **وقت ما بتحصل**، ويحسبها بدقة تاني يوم الصبح. بيمشي مع العميل من أول click لحد تقرير الصبح.

بنيناه بـ Kafka و Spark و Delta و Databricks و Postgres و Grafana و Power BI.

> طيب، قبل ما نشوف إزاي بيشتغل، خلونا نفهم المشكلة بالظبط. المتجر عنده تلات أسئلة.

## سلايد 2: المشكلة

أي متجر أونلاين عنده تلات أسئلة مش عارف يجاوب عليهم.

- **أولًا: أنهي إعلانات بتكسّبنا فعلًا؟** زي ما شفنا في حملة eid_sale_1: 21.55 على الورق، و 11.67 في الحقيقة. السبب إن الأوردرات كاش عند الاستلام، وجزء منها بيتلغي. فلما نبص على المبيعات "المتسجلة"، بتبان أحسن من الحقيقة.
- **ثانيًا: العملاء بيسيبوا فين؟** وده بيجي بعد الإعلان، لأن لما العميل يدخل، بيعدّي على مشاهدة المنتج، وبعدها الـ cart، وبعدها الـ checkout، وبعدها الأوردر. أنهي خطوة بتخسّرنا أكتر ناس؟
- **ثالثًا: إيه اللي محتاج تصرف دلوقتي؟** وده الـ click اللي وصل لرف فاضي: أنهي منتج خلص من المخزن؟ وهل الإعلانات لسه بتبعت ناس له؟

والصعوبة إن الإجابات دي موجودة في أنظمة مختلفة، وعلى توقيتات مختلفة. فريق العمليات محتاج الإجابة **دلوقتي**، وفريق التسويق محتاجها **كل صبح**.

> يعني مفيش نظام واحد يخدم الفريقين. وده بالظبط اللي حاولنا نحله.

## سلايد 3: الحل

فكرة الحل عندنا بسيطة: **مصدر بيانات واحد، ومسارين، لفريقين**.

بنجمع كل حاجة بتحصل: click وأوردر وتوصيل وإعلان وحركة مخزون. وبننضّفها **مرة واحدة**، فيطلع عندنا نسخة واحدة موثوقة. ومن النسخة دي بنفتح مسارين:

- **المسار اللحظي (Live lane):** في أقل من دقيقة. ده للعمليات، لأنهم محتاجين يتصرفوا دلوقتي. بيشوفوا في Grafana المنتجات اللي خلصت، والطلب اللي ضاع، والتوصيلات المتأخرة، وفي alerts بتنبّههم.

- **المسار الصباحي (Batch lane):** كل صباح. ده للتسويق، لأنهم محتاجين أرقام متأكد منها قبل ما ياخدوا قرار الميزانية. بيقروها في Power BI.

ولأن المسارين بيستخدموا **نفس البيانات**، حصلت حاجة مهمة:

> قدرنا نحسب كام جنيه من ميزانية الإعلانات راح لمنتجات كانت خالصة من المخزن. في الـ demo بتاعنا طلع حوالي **10%** من الميزانية، والمبيعات المعرّضة للخطر حوالي **71 ألف جنيه**. وكمان بنقيس الـ ROAS على الأوردرات المتسلّمة، مش المتسجلة، فنوصل للرقم الحقيقي.

طيب إزاي ده بيحصل من جوه؟ تعالوا نبص على الـ architecture.

## سلايد 4: الـ Architecture

هنمشي مع البيانات من الشمال لليمين، زي ما العميل بيمشي.

**البداية:** عندنا **Simulator** بيولّد events واقعية: أحداث التطبيق والأوردرات والتوصيلات والمخزون لـ 8 dark stores. وكمان **Mock ad API** بيمثّل حملات Meta و Google.

الأحداث دي كلها بتدخل على **Kafka**، وفيه 4 topics. وجوه Kafka الرسايل بتفضل محفوظة حتى لو المعالجة وقفت، فمفيش حاجة بتضيع.

ومن Kafka بيستلمها **Spark Structured Streaming**، وبيشتغل جوه Docker. وبيمرّرها على تلات خطوات:

- **Bronze:** الرسايل الخام زي ما جت.
- **Silver:** بيانات متحقق منها ومتشالش منها التكرار.
- **Quarantine:** الصفوف المرفوضة، ومعاها سبب الرفض.

وعند Silver بيحصل التفرع اللي اتكلمنا عنه:

- **المسار اللحظي:** من Silver للـ Postgres، وفيه 16 جدول live و serving. ومنها لـ **Grafana**: 6 dashboards و 8 alerts.

- **المسار الصباحي:** من Silver لـ **Databricks**، وبنبني فيها **Gold** على شكل star schema، وبنشغّل عليها 66 data quality check. وبعدها **Power BI** بـ 7 صفحات و 103 measure. وبرضه **Airflow** بيشغّل تقرير الإعلانات كل يوم الساعة 3 الصبح UTC.

وأخيرًا نقطة مهمة: كل اللي على شمال Databricks بيشتغل على لاب توب Windows واحد جوه Docker. و Gold بيتكتب **بس** من Databricks build.

> دي الصورة الكاملة. دلوقتي هنفتح كل layer ونشوف جواها إيه. وهسلّم الكلمة لرفيدة، وهتبدأ بأول خطوة في الرحلة: إزاي بنجمع البيانات، وهي الـ **Collect**.